TEZ NOTLARI

1.AITEX:

- https://www.aitex.es/afid/

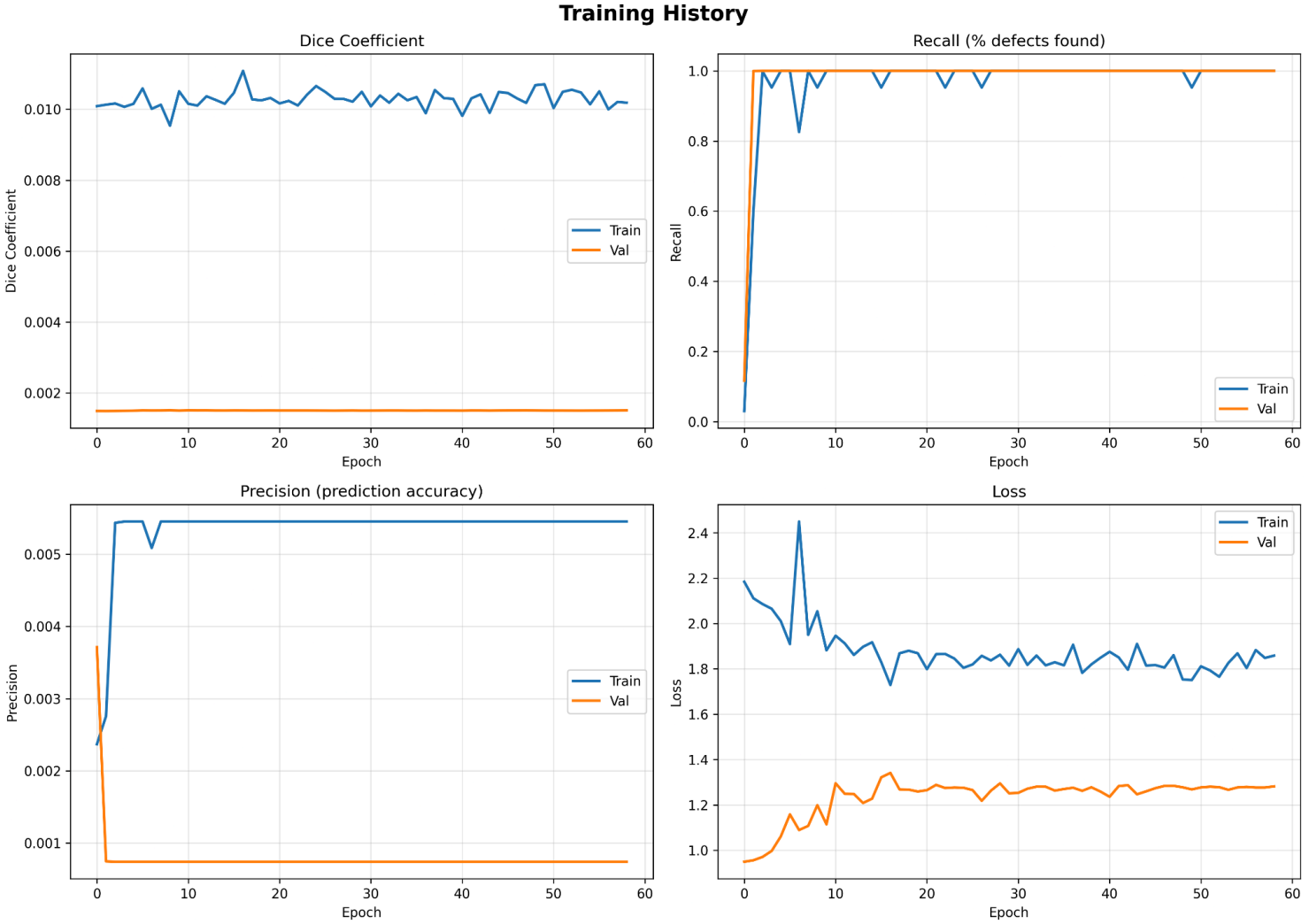
Aspect	Finding	Meaning
Folder structure	3 main folders: Defect, NoDefect, Mask	We have both classification and segmentation possibilities
Subfolders	7 subfolders under NoDefect_images	Probably different fabric batches or materials
Class counts	106 defect / 141 no defect	Nicely balanced classes
Image sizes	~4096×256 pixels	Long, horizontal texture images
Color mode	`'L'` or `'LA'`	Grayscale images (1 channel, sometimes with alpha)

- **Approaches**
    1. Binary Classification (defect vs no-defect)
    2. Semantic Segmentation (pixel-level defect localization)
- **Analysis of AITEX Dataset (AITex_detailed_analysis.py)**
    1. Defect sizes: Mean 0.45% of image, but many are <5%
    2. Image quality: Brightness and contrast distributions
    3. Class imbalance: 141 no-defect vs 106 defect images
    4. Verified that masks correctly align with defect images (AITex_mask_checking.py)
- **Dataset Preparation (AITex_datasetprep.py)**
    1. Collected all defect images (106 images) and no-defect images (141 images)
    2. Split them 80/20 into training (85 defect + 112 no-defect = 197) and validation (21 defect + 29 no-defect = 50)

3. Organized them into aitex_dataset_processed/train/ and val/ folders with Defect/ and NoDefect/ subfolders

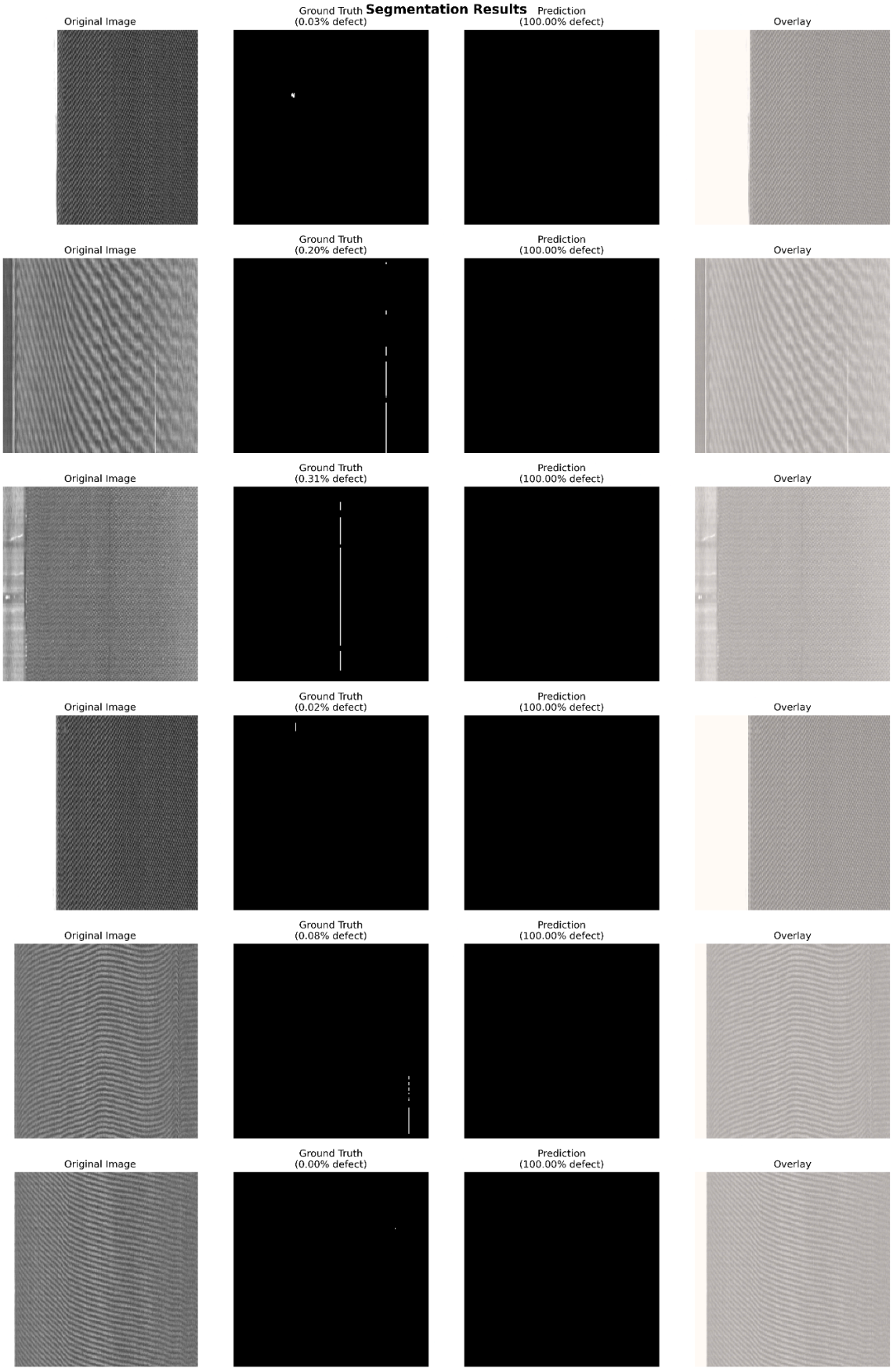
- **Preprocessing**
  - Original size: 4096×256 (long horizontal strips)
  - Resized to: 224×224 (classification) / 256×256 (segmentation)
  - This resizing may lose fine defect details
- **Classification**
  1. **Simple CNN (AITex_classification_train.py)**
     - Built a basic 3-layer CNN from scratch
     - Trained to classify entire images as "Defect" or "NoDefect"
     - Used for binary classification (yes/no defect present)
  2. **Transfer Learning (AITex_classification_pretrained.py)**
     - Used MobileNetV2 (pre-trained on ImageNet) as a feature extractor
     - Added custom classification head
     - Included data augmentation (flips, rotations, zoom)
     - Analyzed misclassified images

- **Segmentation (AITex_unet_segmentation.py)**
  1. Built a U-Net model to:
  - Take textile images as input
  - Output a pixel-by-pixel mask showing exactly where defects are
  - This is much more useful than classification because it localizes defects



- Dice Coefficient Graph:
  - Flat line at ~0.01. No learning happening.
  - No gap between train/val. Not overfitting, just not learning at all
  - Conclusion: Task is too hard for this approach
- Recall Graph:
  - Jumps to 100% immediately. Model learns to predict everything as defect
  - Stays at 100%. This strategy works for maximizing recall
  - Problem: Recall alone is meaningless if precision is 0%

- Precision Graph:
    - Stays near 0%. Every pixel flagged means 99.45% false positives
    - No improvement. Model never learns to discriminate
    - Training vs validation gap: Slight overfitting, but both are terrible
- Loss Graph:
    - Training loss decreases. Model optimizing weighted loss
    - Validation loss plateaus, but not learning generalizable patterns
    - Gap between curves: Some overfitting, but main issue is approach failure


Segmentation Results
Original Image
Ground Truth
(0.03% defect)
Prediction
(100.00% defect)
Overlay
Original Image
Ground Truth
(0.20% defect)
Prediction
(100.00% defect)
Overlay
Original Image
Ground Truth
(0.31% defect)
Prediction
(100.00% defect)
Overlay
Original Image
Ground Truth
(0.02% defect)
Prediction
(100.00% defect)
Overlay
Original Image
Ground Truth
(0.08% defect)
Prediction
(100.00% defect)
Overlay
Original Image
Ground Truth
(0.00% defect)
Prediction
(100.00% defect)
Overlay

- Image 1:
  - Ground Truth: 0.03% defect (19 pixels - tiny spot)
  - Prediction: 100% defect (65,536 pixels - entire image!)
  - Dice Score: 0.001

- Image 2:
  - Ground Truth: 0.20% defect (132 pixels - small line)
  - Prediction: 100% defect (entire image)
  - Dice Score: 0.004

- **Class Imbalance**
  - **Image-Level (Classification)**
    141 no-defect : 106 defect images
    Ratio: 1.33:1
  - **Pixel-Level (Segmentation)**
    Background pixels: 99.45%
    Defect pixels: 0.55%
    Ratio: ~182:1

- **Main Problem**

- The U-Net segmentation approach failed due to the combination of extreme class imbalance (182:1), very small defect sizes (0.55% of pixels), and insufficient training data (84 images).
- The segmentation model is predicting entire images as defects because:
  1. Defects are extremely small (0.55% of pixels on average)

2. The weighted loss over-corrected - it's better to predict everything as defect and catch all real defects than to miss any
3. Not enough training data - 84 images isn't much for learning tiny, subtle patterns